POO - Programação Orientada a Objetos

Classes Abstratas e Interfaces

Prof. Msc Marcos de Souza

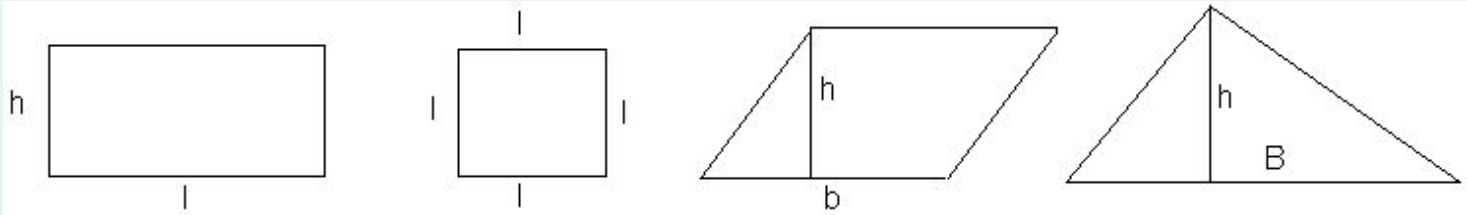
Classes Abstratas

- Existem ocasiões nas quais criamos uma classe e percebemos que no futuro outras classes poderão herdar da classe que estamos criando.

- Imagine que ao criar uma classe você perceba que ao invés dela servir a um único propósito ela pode ser generalizada e usada para outros objetivos.

Classes Abstratas

- Exemplo: uma classe para efetuar cálculos com **retângulos, quadrados, paralelogramos e triângulos**:



h – altura l – lado b – base

Classes Abstratas

- A primeira coisa que vem à cabeça é que a tal classe poderia ser usada com diferentes tipos de figuras geométricas.

- Logo ao invés de criar uma classe para cada tipo de figura geométrica você poderia criar uma classe generalizada na qual todas as figuras pudessem se basear.

Classes Abstratas

- As figuras geométricas não têm muita coisa em comum; pois enquanto o triângulo e o paralelogramo possuem uma base, o retângulo e o quadrado não possuem.

- O que elas têm em comum: todas possuem um nome e uma descrição.

Classes Abstratas

- Podemos então criar uma classe que apenas defina as características que sejam comuns de forma a deixar a implementação de cada uma delas para cada objeto.

- A classe genérica servirá como base para a criação de outras classes, nela deveremos apenas declarar os métodos que são comuns.

Classes Abstratas

- A classe abstrata é sempre uma superclasse que não possui instâncias.
- Ela define um modelo (*template*) para uma funcionalidade e fornece uma implementação incompleta.
- Cada uma das classes derivadas completa a funcionalidade da classe abstrata adicionando um comportamento específico.

Classes Abstratas

- Usamos a palavra-chave *abstract* para impedir a instanciação de uma classe.

- Normalmente têm métodos abstratos:
  - método sem corpo definido
  - implementação é deixada para as subclasses
  - sintaxe da declaração:

```
public abstract void debitar(double valor);
```

- Subclasses *não abstratas* têm obrigatoriedade de implementar os métodos abstratos das superclasses.

Classes Abstratas

- Os métodos abstratos são implementados nas classes derivadas concretas com o objetivo de definir o comportamento específico.

- O método abstrato define apenas a assinatura do método e, portanto, não contém código.

Classes Abstratas

- Regras sobre métodos abstratos e as classes que os contêm:
  - Toda classe que possui métodos abstratos é automaticamente abstrata.
  - Uma classe pode ser declarada abstrata mesmo se ela não possuir métodos abstratos.
  - Uma subclasse de uma classe abstrata pode ser instanciada se:
    - Sobrescreve cada um dos métodos abstratos de sua superclasse;
    - Fornece uma implementação para cada um deles.

Classes Abstratas

- Regras sobre métodos abstratos e as classes que os contêm:
  - Se uma subclasse de uma classe abstrata não implementa todos os métodos abstratos que ela herda, então a subclasse também é abstrata.
  - Para declarar um método abstrato coloque a palavra-chave *abstract* no início da declaração do método.
  - Para declarar uma classe abstrata coloque a palavra-chave *abstract* no início da definição da classe.

Classes Abstratas

```
public abstract class Conta {
   protected double saldo;

   public void retira(double valor) {
      this.saldo -= valor;
   }

   public void deposita(double valor) {
      this.saldo += valor;
   }

   public double getSaldo() {
      return this.saldo;
   }

   public abstract void atualiza();
}
```

Classes Abstratas - Vantagens

- Conceito que **agrupa categorias que possuem apenas algumas características em comum**
- Melhor estruturação do software
- Melhor reuso
- Melhor extensibilidade

Interfaces

- Como criar a classe **CirculoDesenhavel** que é ao mesmo tempo **Circulo** e **Desenhavel**?

```
public class CirculoDesenhavel
      extends Circulo, Desenhavel { ... }
```

- Mas o código acima está ERRADO! Pois Java não suporta herança múltipla.

Interfaces

- Para resolver esse tipo de problema, Java define o conceito de *interfaces.*

- Uma interface é semelhante a uma classe abstrata, exceto pelo fato de usar a palavra-chave *interface* no lugar de *abstract class* na definição da classe.

Interfaces

- **Interface** é um contrato onde quem assina se responsabiliza por implementar todos os métodos definidos na própria interface.

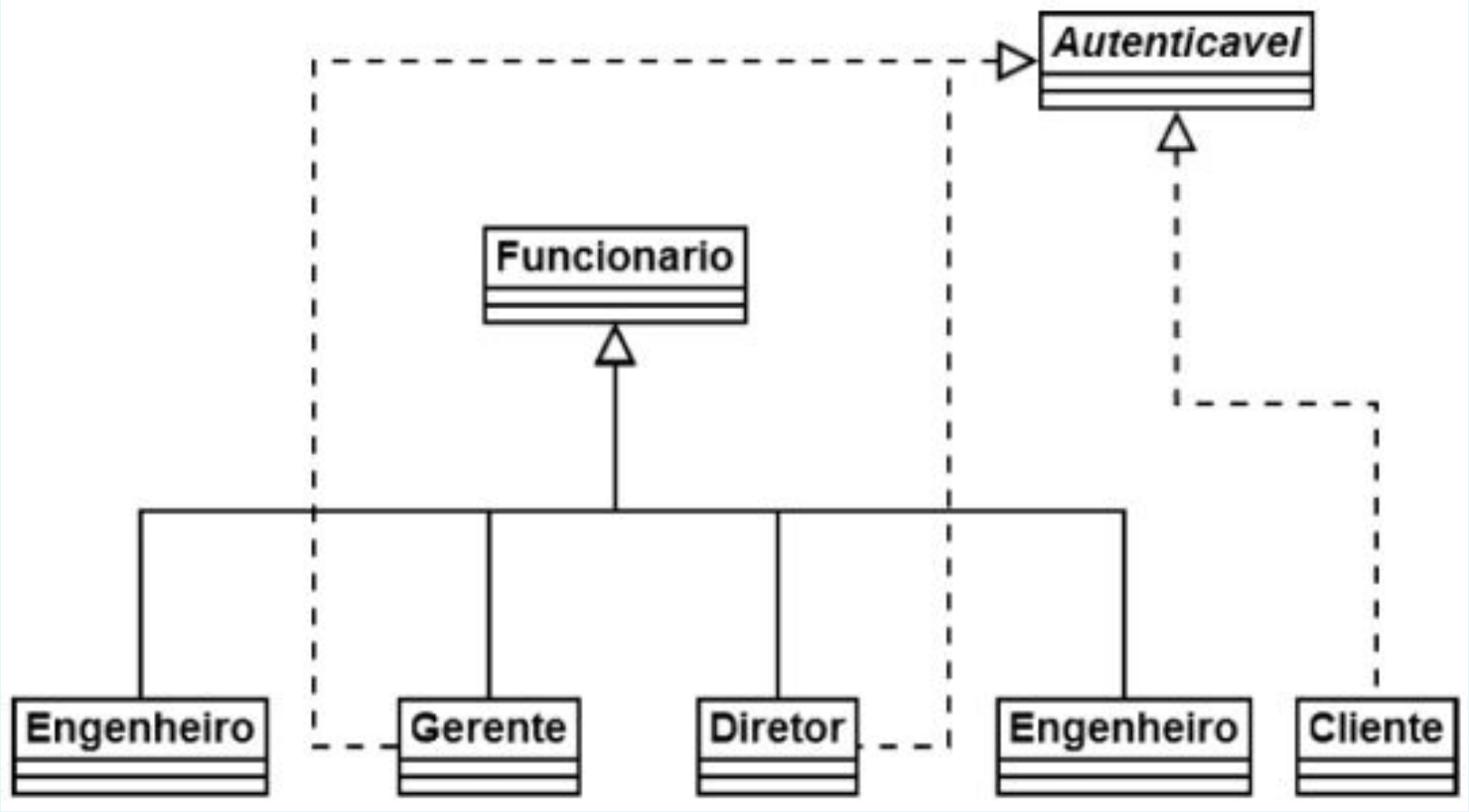
- Ela só expõe o que o objeto deve fazer e não como ele faz. Como ele faz será definido em uma implementação dessa interface.

Classes Abstratas X Interfaces

- Declaração de métodos:
  - Em uma classe abstrata podem ser definidos métodos abstratos e não-abstratos;
  - Todos os métodos de uma interface são implicitamente abstratos.

- Declaração de variáveis:
  - Em uma classe abstrata podem ser definidas variáveis de instância, classe e constantes;
  - Todos os atributos de uma interface são sempre **public static final** (constantes).

Interfaces


Autenticavel
Funcionario
Engenheiro
Gerente
Diretor
Engenheiro
Cliente

Interfaces

```
public interface Autenticavel {
   public boolean autentica(int senha);
}
```

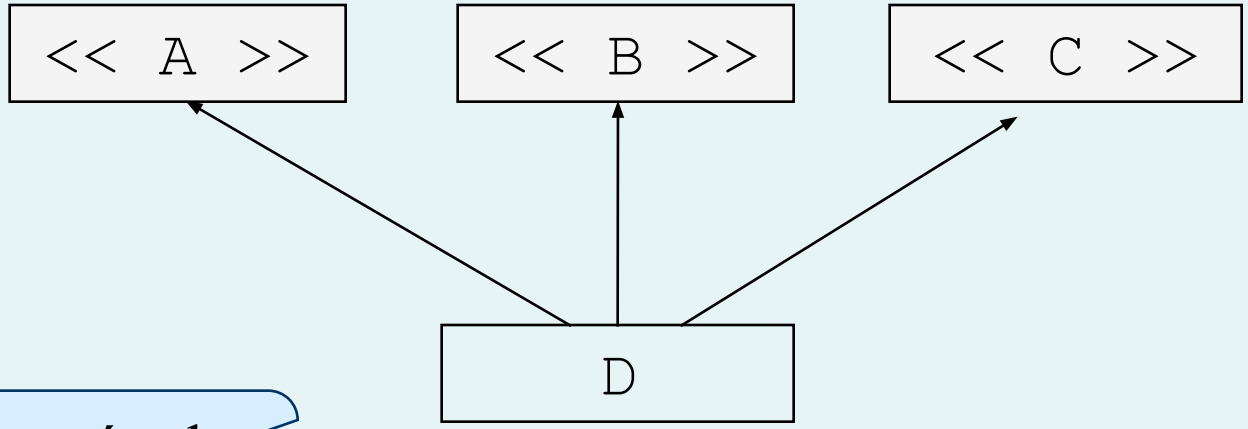
```
class Gerente extends Funcionario
                        implements Autenticavel {
   private int senha;

   // outros atributos e métodos
   public boolean autentica(int senha) {
      if (this.senha != senha) {
         return false;
      }
      // pode fazer outras possiveis verificacoes,
      // como saber se esse departamento do
      // gerente tem acesso ao Sistema
      return true;
   }
}
```

Interfaces

- Introduz conceitos de subtipo e supertipo (ao invés de subclasse e superclasse)
- Os métodos são implementados pelos subtipos
- Não possuem construtores -> Não podem ser instanciadas

Interfaces - Herança

- Uma classe pode implementar mais de uma interface (pode ter vários supertipos)



Implementa os métodos de A, B e C

Interfaces - Herança

- Diferente das classes concretas, uma interface pode herdar de mais de uma superclasse.
- Você não herda métodos e atributos, e sim responsabilidades.

Relação de implementação

- A classe que implementa uma interface deve definir os seus métodos:
  - classes concretas têm que implementar todos os métodos da interface;
  - classes abstratas que implementam interfaces podem deixar de implementar algum método abstrato definido na interface.

Subtipos e instanceof

```
public class Circulo extends Forma implements
   Selecao, Rotacao, Movimentacao {
   ...
}
```

```
...
Circulo c = new Circulo();
res1 = c instanceof Circulo;
res2 = c instanceof Selecao;
res3 = c instanceof Rotacao;
res4 = c instanceof Movimentacao;
...
```

Interfaces e atributos

- Interfaces são excelentes repositórios para constantes globais!

```
public interface TipoConta{

 public static final int CONTA = 0;
 public static final int POUPANCA = 1;
 public static final int BONIFICADA = 2;
 public static final int IMPOSTO = 3;

}
```

Perguntas?